

RIGHT CONTEXT

LLM 에이전트가 일을 끝낼 수 있도록
필요한 도구와 문서를 찾아 전달하는 AI 플랫폼 엔지니어

손성준 AI Platform Engineer · Part Leader

GitHub SonAlengine Blog infoedu.co.kr Email sonsj97@gmail.com

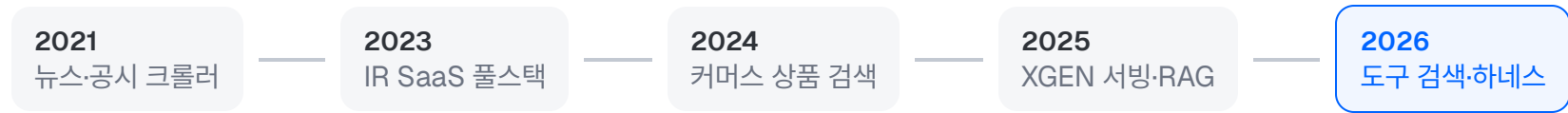
Portfolio 2026

AI Platform Engineer & Part Leader, 손성준입니다

2021년 뉴스·공시 크롤러로 시작해 커머스 상품 검색, 사내 문서 RAG를 거쳐 지금은 LLM 에이전트가 쓸 도구와 맥락을 찾는 일을 하고 있습니다. 찾는 대상은 바뀌었지만, 필요한 것을 정확히 골라 전달한다는 문제는 같았습니다.

플래티어 AI R&D팀에서 8명 기술·컨설팅 파트를 이끌며 XGEN의 배포 플랫폼과 모델 서빙을 맡고, 고객 현장에서는 에이전트를 업무에 붙이는 FDE 역할을 합니다.

재현되지 않은 개선은 주장하지 않습니다.



경력

(주)플래티어

AI R&D팀 기술·컨설팅 파트장 · 검색에서 AI 플랫폼으로
2024년 3월 ~ 2년 7개월

서울아이알네트워크

플랫폼사업부 · Fullstack
2023년 2월 ~ 2024년 2월 1년 1개월

에이치제이브레인 · 데이터 수집
2021년 7월 ~ 2022년 12월 1년 6개월

학력

고려대학교 SW·AI융합대학원

인공지능융합학과 석사과정
2026년 3월 ~

한양대학교

도시공학과
2023년 2월 졸업

검색 엔지니어로 시작해 AI가 쓸 도구와 정보를 찾아 주기까지 해온 일들입니다

[분류](#)[내 역할](#)[주제](#)

고객 현장에 AI 에이전트 붙이기

FDE · 고객사 30여 곳 시연·PoC

[회사](#)[FDE · 파트장](#)[고객 현장](#)

에이전트가 일을 끝내게 만드는 실행 통제

작업당 AI 호출 횟수 절반

[회사](#)[평가 설계](#)[실행 통제](#)

AI 에이전트 플랫폼 XGEN 배포와 모델 운영

8명 파트 리딩 · 사내망 운영

[회사](#)[파트 리더](#)[인프라](#)

한국어 LLM과 결정 모델 파인튜닝

POLAR 14B · SFT·DPO · LoRA

[회사 · 개인](#)[SFT 담당](#)[학습](#)

API 1,000개 중 일에 필요한 것만 골라 주기

graph-tool-call 오픈소스

[오픈소스](#)[단독 개발](#)[공개 배포](#)

쇼핑몰 상품 검색 키워드와 의미를 함께

상품 검색 · 14개월

[회사](#)[쿼리 설계](#)[검색](#)



30여 곳 5곳 500명 5회

XGEN 시연·PoC를
맡은 고객사

파트가 맡은 PoC 중
계약으로 이어진 곳

지방은행 플랫폼
사용자 규모

현장
사용자 교육

고객 현장 FDE

고객사에 들어가 AI 에이전트를 업무에 붙인다

파트장으로 고객사 시연과 PoC를 이끌어 30여 곳 중 5곳을 계약으로 이어 갔고, 지방은행 구축에는 직접 FDE로 들어가
요구를 듣고 에이전트를 만들어 고객 서버에 배포한 뒤 사용자 교육까지 맡았습니다. 배포 플랫폼과 모델 서빙을 만든 경험이
현장에서 막히는 문제를 제품 쪽에서 바로 고치는 바탕입니다.

2026년 6월 조직개편 이후 기술·컨설팅 파트장

회사

FDE · 파트장

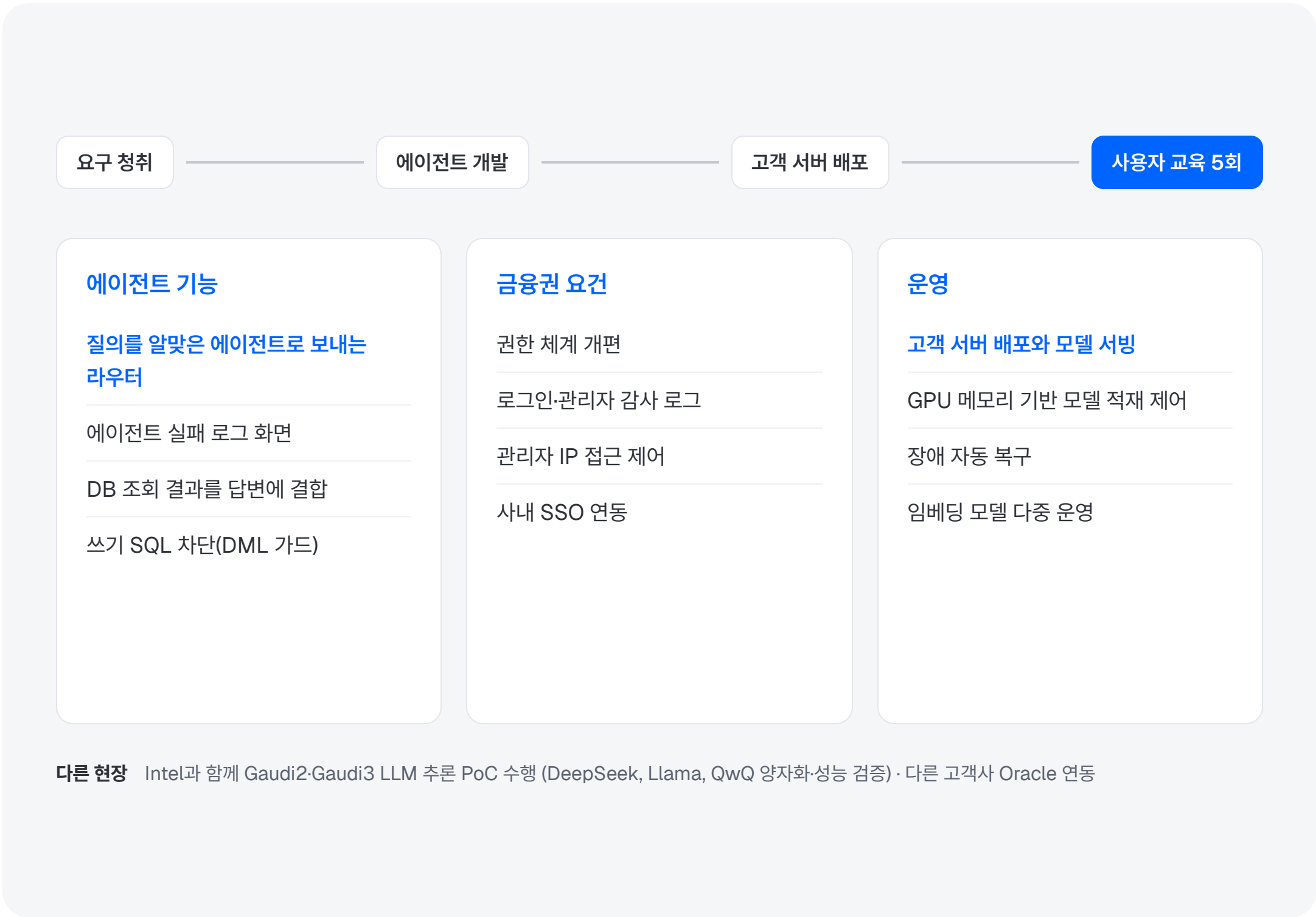
고객사명은 비공개

지방은행 현장에서 요구를 기능으로

지방은행 생성형 AI 플랫폼 구축에 पार्ट장이자 FDE로 들어가, 에이전트는 직접 만들고 금융권 요건과 운영은 파트원들과 나눠 구축했습니다. 고객 커뮤니케이션과 사용자 교육은 제가 맡았습니다.

사용자 500명, 동시 사용자 10명 규모의 서비스입니다. 금융권이라 기능보다 먼저 권한, 감사, 접근 제어 요건을 맞췄습니다.

고객사명과 서버 사양은 비공개 · 파란 표시는 직접 구현, 나머지는 파트 구현



8곳

Helm 차트 하나로 운영하는
서버 환경

10종

지원하는 AI 모델
실행 엔진

7종

올릴 수 있는 모델 종류
대화·음성·문서 인식 등

90%

배포 시스템 커밋 중
내 커밋 비율

XGEN · 배포 플랫폼과 모델 서빙 담당

하나의 코드로 여러 서버에 XGEN 올리기

XGEN은 워크플로·문서·MCP·모델 서빙으로 구성된 사내 AI 에이전트 플랫폼입니다.

앱 서비스는 파트원들이 나눠 만들고, 저는 8명 파트를 이끌면서

이 서비스들이 고객사 서버와 폐쇄망(인터넷이 차단된 사내망)에서 같은 방식으로 배포·운영되도록 배포 플랫폼과 모델 서빙을 맡았습니다.

2025년 6월 ~ 현재

회사

파트 리더

배포 플랫폼 · 모델 서빙 직접

밖에서 본 XGEN

XGEN은 기업이 자기 환경에 맞는 AI 에이전트를 설계부터 운영까지 관리하는 에이전틱 AI 플랫폼입니다.

2026년 7월 TTA 소프트웨어 품질인증 GS 1등급을 받았고, 온프레미스와 클라우드를 모두 지원합니다. 금융·커머스 고객사에 공급되고 있습니다.

보도는 회사 발표이고, 아래 파란 줄은 그중 제가 직접 맡은 부분입니다.

출처 · 아시아경제 2026-07-08 · 뉴스핌 2026-08-06 · EBN 2026-09 · 사진 플래티어 제공

언론 보도

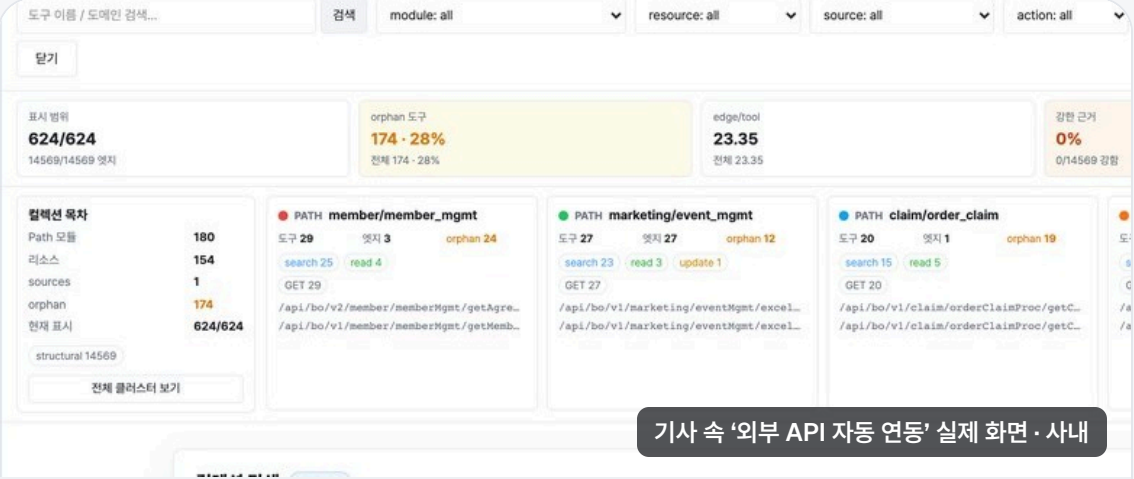


에이전틱 AI 플랫폼
XGEN
GS 인증 1등급 획득

1등급
GOOD Software

아시아경제 · 2026.07.08
플래티어 ‘엑스젠(XGEN)’, GS인증 1등급 획득

내 뉘 · GS 성능시험 서버 환경과 원데이터 구축



기사 속 ‘외부 API 자동 연동’ 실제 화면 · 사내

뉴스핌 · 2026.08.06
플래티어, ‘엑스젠’ 운영·신뢰·확장성 강화

내 뉘 · 온프레미스 배포 플랫폼, 외부 API 자동 연동



EBN · 2026.09
플래티어, 금융 테크 컨퍼런스서 에이전틱 AI 플랫폼 ‘엑스젠’ 소개

회사 발표 · 제품 소개 행사

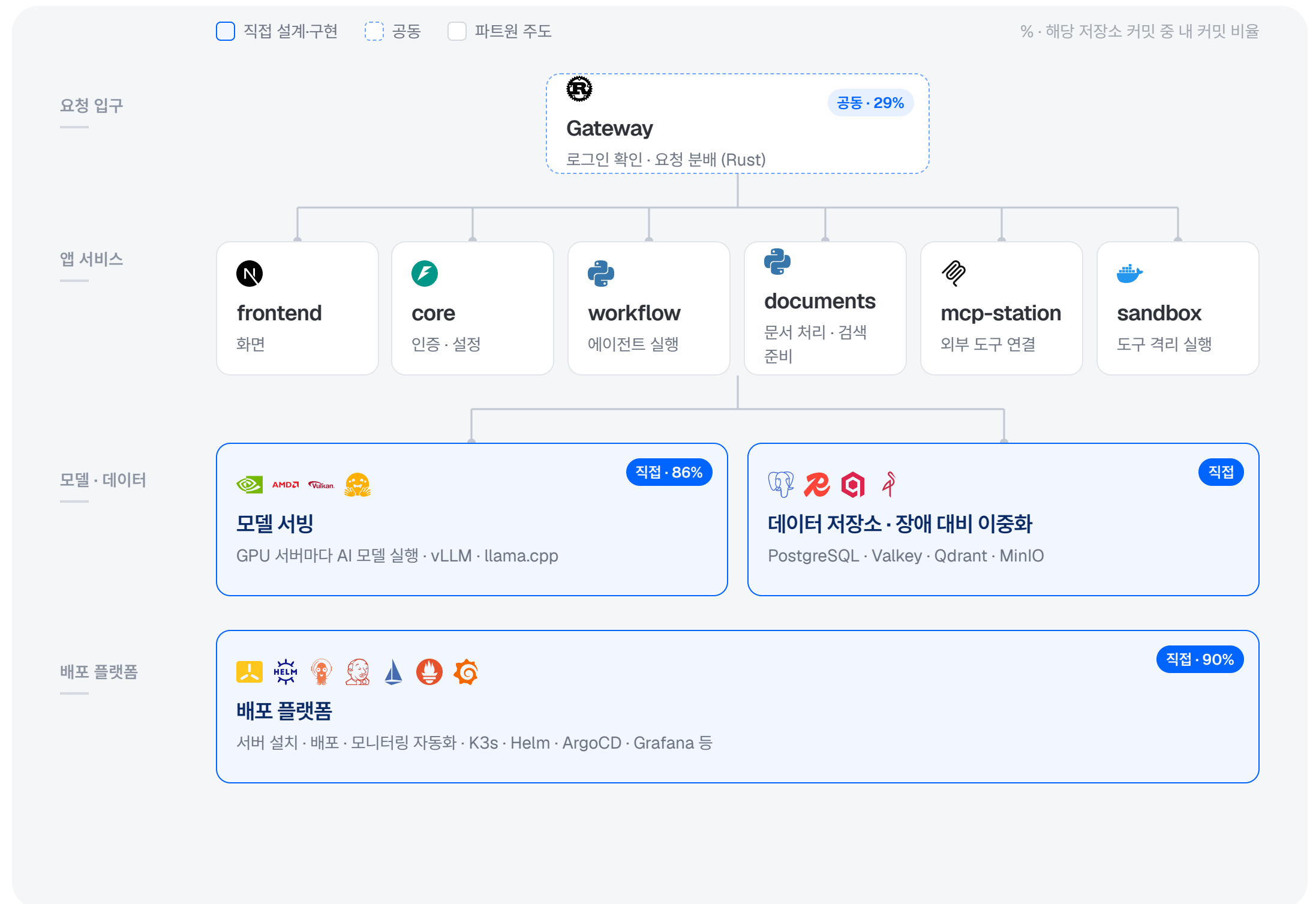
XGEN 구성과 제가 맡은 영역

XGEN은 게이트웨이 뒤에 6개 앱 서비스와 모델 서빙, 데이터 계층이 붙은 구조입니다. 앱 서비스는 파트원들이 도메인별로 나눠 소유합니다.

저는 파트를 이끌면서 이 서비스들을 어느 서버에서든 같은 방식으로 올리는 **배포 플랫폼**과 GPU에 모델을 올리는 **모델 서빙**을 직접 설계하고 대부분 작성했습니다.

게이트웨이는 파트원이 시작한 저장소에 제가 인증·라우팅을 작성한 공동 작업입니다.

근거 · 배포 저장소 커밋 962/1,069 · 모델 서빙 저장소 커밋 189/218 (2026-09-26 집계)



1.0에서 폐쇄망 운영까지

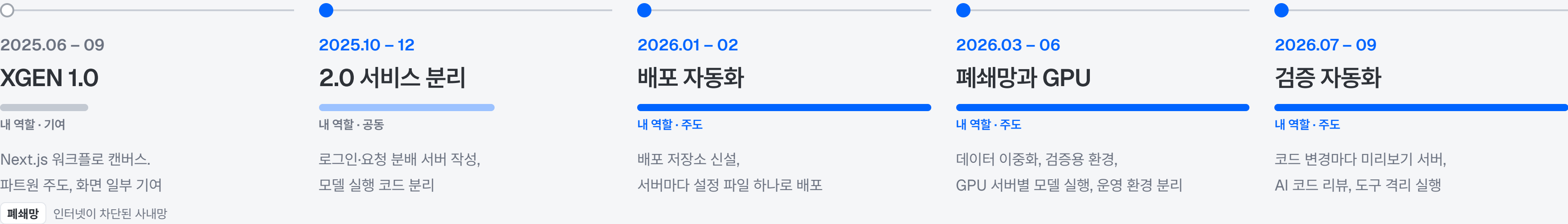
처음부터 인프라를 맡은 것은 아닙니다. XGEN 1.0은 파트원들이 주도했고, 저는 RAG·서빙 설정 화면 일부를 만들었습니다.

2.0에서 서비스를 나누면서 게이트웨이 인증·라우팅과 모델 서빙 분리를 맡았고, 2026년 1월 배포 저장소를 새로 만들면서 배포 플랫폼 전체를 책임지게 됐습니다.

이후 고객사 서버, 폐쇄망, GPU 종류가 늘어날 때마다 앱 코드를 바꾸지 않고 설정 파일만 추가해 올릴 수 있는 쪽으로 구조를 다듬었습니다.

최근에는 사람이 모든 MR을 볼 수 없는 규모가 되어, 검증을 파이프라인에 넣는 작업을 하고 있습니다.

단계별 변화



서버마다 다른 환경을 하나의 차트로

XGEN은 서버마다 배포 도구(ArgoCD)가 따로 있고, 운영 서버는 사람이 확인한 뒤, 개발 서버는 자동으로 변경을 반영합니다.

처음에는 ArgoCD ApplicationSet으로 앱 6개를 하나로 묶었습니다. 하지만 이 방식은 배포 도구 하나가 모든 서버를 관리한다는 전제였고, 서버마다 반영 규칙을 나누기 어렵고, 문제가 생기면 생성 규칙 자체를 고쳐야 했습니다.

한 달 만에 되돌리고, 하나의 Helm 차트에 서버별 설정 파일만 두는 구조로 바꿨습니다. 지금은 고객사 3곳과 사내 5개 환경, 모두 8개가 같은 차트를 씁니다.

근거 · 커밋 기록 · 2026-01-17 도입 → 2026-02-17 ~ 18 전환



AMD GPU에서 모델이 로드 직후 멈춘 문제

직접 디버깅 · 2026-01-31 · 약 6시간

2026-01-31 · 시도 경로

- 1 모델 일부만 GPU에 올리기
실패 · 같은 오류
- 2 검색용 모델은 CPU로 빼기
부분 해결 · 검색용 모델 오류만 사라짐
- 3 메모리 자동 계산 끄기
부분 해결 · 멈춤은 사라졌지만 답이 가끔 깨짐
- 4 고속 연산 기능 끄기
안정화 · 기능을 계속 꺼야 하는 구조
- 5 범용 GPU 방식(Vulkan)으로 전환
해결 · 모델 전체를 GPU에서 실행

증상

AMD GPU에서 모델을 올리자마자 메모리 오류로 멈춤

NVIDIA에서는 같은 설정으로 문제없이 돌던 llama.cpp 서버였습니다. 2026년 1월 31일, 약 6시간에 걸쳐 원인을 좁혔습니다.

시도

설정 옵션을 하나씩 바꾸며 네 번 시도

레이어를 줄여도 같은 오류가 나서 메모리 양의 문제가 아님을 확인했습니다. 메모리 자동 계산을 끄자 멈춤은 사라졌지만, 이번에는 답이 가끔 깨졌습니다.

원인

옵션 문제가 아니라, 이 GPU용 AMD 연산 드라이버(ROCm) 자체가 불안정했다

GPU 메모리 크기를 잘못 읽는 문제와 고속 연산 기능의 오류가 겹쳐 있었습니다. 옵션을 하나씩 끄는 방식으로는 근본적으로 해결되지 않는다고 판단했습니다.

해결

실행 방식을 범용 GPU 방식(Vulkan)으로 바꿔 해결

이후 GPU 종류를 감지해 NVIDIA는 vLLM-CUDA, AMD는 Vulkan, GPU가 없으면 CPU를 자동으로 고르도록 정리했습니다.

근거 · 커밋 기록 2026-01-31 · 블로그 「llama.cpp 서버 운영기」

ROCm AMD 전용 GPU 연산 드라이버 Vulkan 제조사와 상관없이 쓰는 범용 GPU 방식

GPU 서버마다 AI 모델 올리기

직접 구현 · 이 저장소 커밋의 86%

모델 서버가 GPU 서버마다 하나씩 자동으로 뜨도록(DaemonSet) 바꿔,
서버가 늘어도 배포 설정을 고치지 않게 했습니다.

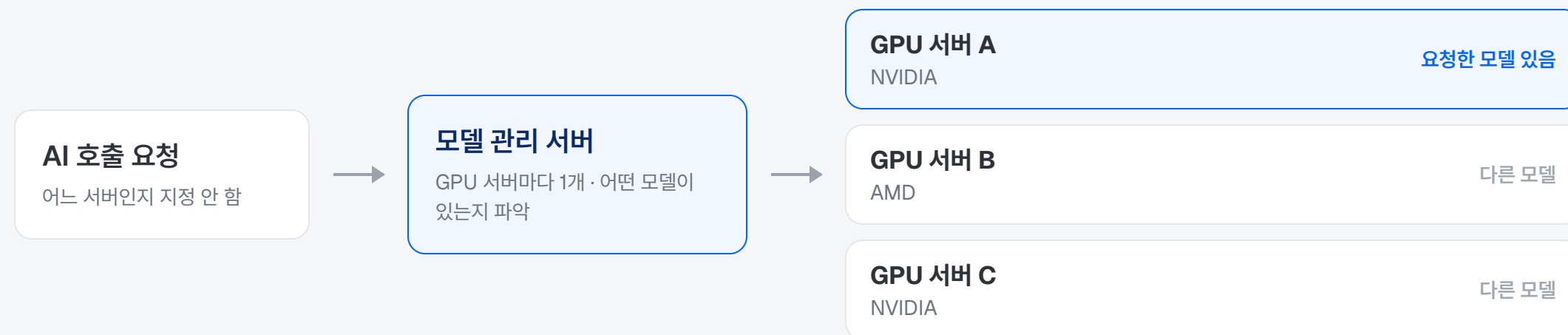
어느 서버에 어떤 모델이 떠 있는지 모아 두고, 서버를 지정하지 않은 AI
호출은 그 모델이 있는 서버로 자동으로 보냅니다.

관리 화면에서 GPU 메모리를 보면서 LLM·임베딩·음성 모델을 올리고, 재기동
·테스트·언로드를 한 곳에서 합니다.

여러 서버에 모델 하나를 나눠 올리는 분산 실행은 아직 하지 않았고, 다음
과제로 남아 있습니다.

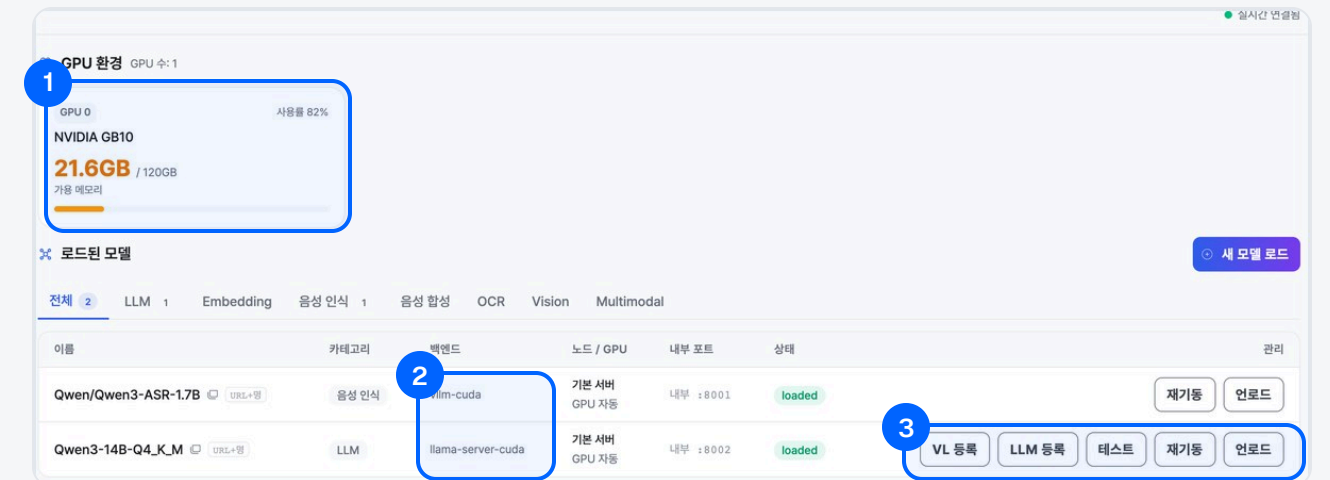
근거 · 커밋 기록 · 2026-05-16 서버별 자동 실행 · 2026-06-22 모델 있는 서버로 전달

요청은 모델이 있는 서버로



모델이 있는 서버로 자동 전달 · 서버가 늘어도 설정 변경 없음 (구성은 예시)

관리 화면 · 사내 개발 서버 (GPU 1대)



- 1 GPU 메모리 보며 모델 올리기
- 2 실행 엔진 자동 선택
- 3 재시작 · 테스트

결과 및 회고

0회

새 서버 환경 6곳을 늘리는 동안
공통 설정을 고친 횟수

-47%

문서 처리 중 최대 메모리
할당 설정 변경 · 2,257 → 1,204MB

3단계

배포 단계
개발 → 검증 → 운영

11개

코드 리뷰 알림을
자동화한 저장소

배운 점 · 줄였다가 되돌린 기록



실사용량 기준으로 줄였다가 OOMKilled가 나서 되돌렸고, 그 과정을 커밋에 남겨 다음 조정의 기준으로 씁니다.

근거 · 커밋 기록 · 2026-02-26 축소 · 2026-02-27 되돌림

리더로서 · 검증을 사람 대신 자동으로



8명이 11개 저장소에 올리는 MR을 한 사람이 다 볼 수 없어, 검증 단계를 파이프라인에 넣었습니다.

근거 · 커밋 기록 · 2026-08-16 미리보기 · 2026-09-07 AI 리뷰 · 2026-09-04 리뷰 알림



57회

9,094

96%

0개

첫 공개 후 6개월간
공개 배포

최근 6개월 다운로드
(PyPI)

전체 커밋 중
내 커밋 비율

설치에 필요한
외부 라이브러리

graph-tool-call

API 1,000개 중 일에 필요한 것만 골라 주기

사내 에이전트에 연결할 API가 1,000개를 넘자, 모든 도구 스키마를 컨텍스트에 넣을 수 없게 되었습니다.
의미 검색으로 도구를 고르면 목표 도구는 찾지만, 그 도구의 입력값을 만들어 줄 도구는 자주 빠졌습니다.
도구 사이의 입력·출력 계약을 그래프로 만들어 둘을 함께 찾는 라이브러리를 설계하고 공개했습니다.

2026년 3월 ~ 현재

- 오픈소스
- 단독 설계·개발
- LLM 도구 검색

환불 도구는 찾지만 주문 조회는 놓친다

“alice@example.com의 주문을 환불해줘”라는 요청에 의미 검색은 `refundOrder` 를 1순위로 찾았습니다.

하지만 `refundOrder` 는 주문 번호가 있어야 실행됩니다. 사용자는 이메일만 알려줬고, 주문 번호를 만들어 주는 `findOrdersByEmail` 은 6개 도구 중 5순위로 밀렸습니다.

상위 3개만 모델에 넘기면 에이전트는 실행 단계에서 멈추거나 주문 번호를 지어냅니다. 그렇다고 전체를 넘기면 도구가 1,000개를 넘는 순간 컨텍스트와 비용이 감당되지 않습니다.

그래서 문제를 “비슷한 도구 찾기”가 아니라 “실행 가능한 묶음 찾기”로 다시 정의했습니다.

의미 검색 순위 · 6개 도구로 줄인 예시 (실제 사내 카탈로그는 1,000개 이상)

1

refundOrder
(order_id)

Refund an existing customer order after its order ID has been resolved.

환불 도구 · 찾음

2

listRefundReasons
() → reason_code

List the supported refund reason codes.

이번 일엔 불필요

3

cancelOrder
(order_id)

Cancel an existing order by order ID.

이번 일엔 불필요

top-3 경계

4

getOrderDetail
(order_id) → payment_id

Read one order by its order ID.

이번 일엔 불필요

5

findOrdersByEmail
(email) → order_id

Find customer orders by email and return matching order IDs.

놓침 · 주문 번호를 찾아 주는 도구

graph-tool-call 결과 주문 조회 → 환불, 두 도구를 순서대로 함께 전달 · AI에 넘기는 도구 설명 분량 **1,476 → 160 토큰**

측정 결과와 제약에서 설계 방향 설정

상위 5개 안에 정답이 든 비율 · API 1,068개 · 질문 50개



Graph를 섞어도 **0%p** 개선 → 순위 매기기 말고 다른 역할이 필요

의미 검색은 별도 AI 모델이 있어야 해서 선택 사항으로 두고, 기본 기능은 설치만으로 동작하게 유지

사실

Graph를 점수에 섞어도 검색이 좋아지지 않았다

9개 데이터셋 공정 비교에서 Graph를 점수에 섞으면 BM25 단독보다 같거나 나빴습니다.

1,068개 도구 기준 Recall@5는 BM25+Graph 78%로 BM25 단독과 같았고, 이 결과를 그대로 공개했습니다.

제약

핵심 검색은 LLM과 외부 의존성 없이 동작해야 한다

폐쇄망과 로컬 모델 환경에서도 설치 즉시 쓰려면 코어는 Python 표준 라이브러리만 써야 했습니다.

가설

Graph는 순위 대신, 빠뜨린 준비 단계 도구를 채우는 데 쓴다

순위는 BM25와 선택적 임베딩에 맡기고, Graph는 목표 도구의 입력값을 만드는 도구를 계약 근거가 있을 때만 추가합니다.

목표

해야 할 일과 그 준비 단계 도구를 함께, 필요한 만큼만 전달한다

검색 결과가 곧 실행 순서가 되도록 만들어, 에이전트가 주문 번호 같은 값을 추측하지 않게 합니다.

Graph를 순위에서 빼고 빠진 후보만 채우게

처음에는 Graph를 BM25-임베딩과 같은 점수 채널로 섞었습니다. 공정 비교에서 이 방식이 BM25 단독보다 나빠지는 것을 확인하고 구조를 바꿨습니다.

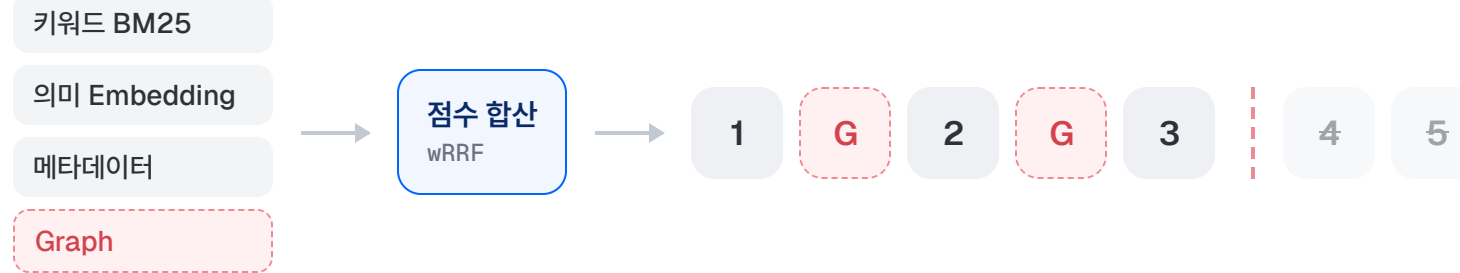
이제 순위는 BM25와 선택적 임베딩이 정하고, Graph는 BM25가 놓친 후보를 뒤에 추가만 합니다. 그래서 **BM25+Graph**는 **BM25** 단독보다 나빠질 수 없습니다.

같은 변경에서 GitHub API에만 맞춰 두었던 별칭 규칙 49개를 지우고, 도구 이름과 설명을 역색인하는 범용 방식으로 바꿨습니다.

근거 · commit 1908eca (2026-03-22)

변경 전 Graph 점수를 다른 점수와 섞음

키워드 검색보다 나빠짐



Graph 후보가 끼어들어 원래 좋은 결과를 밀어냄

변경 후 Graph는 빠진 후보를 뒤에 붙이기만

키워드 검색보다 나빠지지 않음



기존 순위는 그대로 두고, 놓친 후보만 뒤에 추가

1 기존 검색 결과 G Graph가 찾은 후보

BM25 키워드 일치 점수 wRRF 여러 검색 점수를 순위로 합치는 방식 G Graph가 찾은 후보

근거 · commit 1908eca (2026-03-22) · 블로그 「v0.15: 1068 Tool 스트레스 테스트」

필요한 도구는 다 찾고 넘기는 양은 줄였다

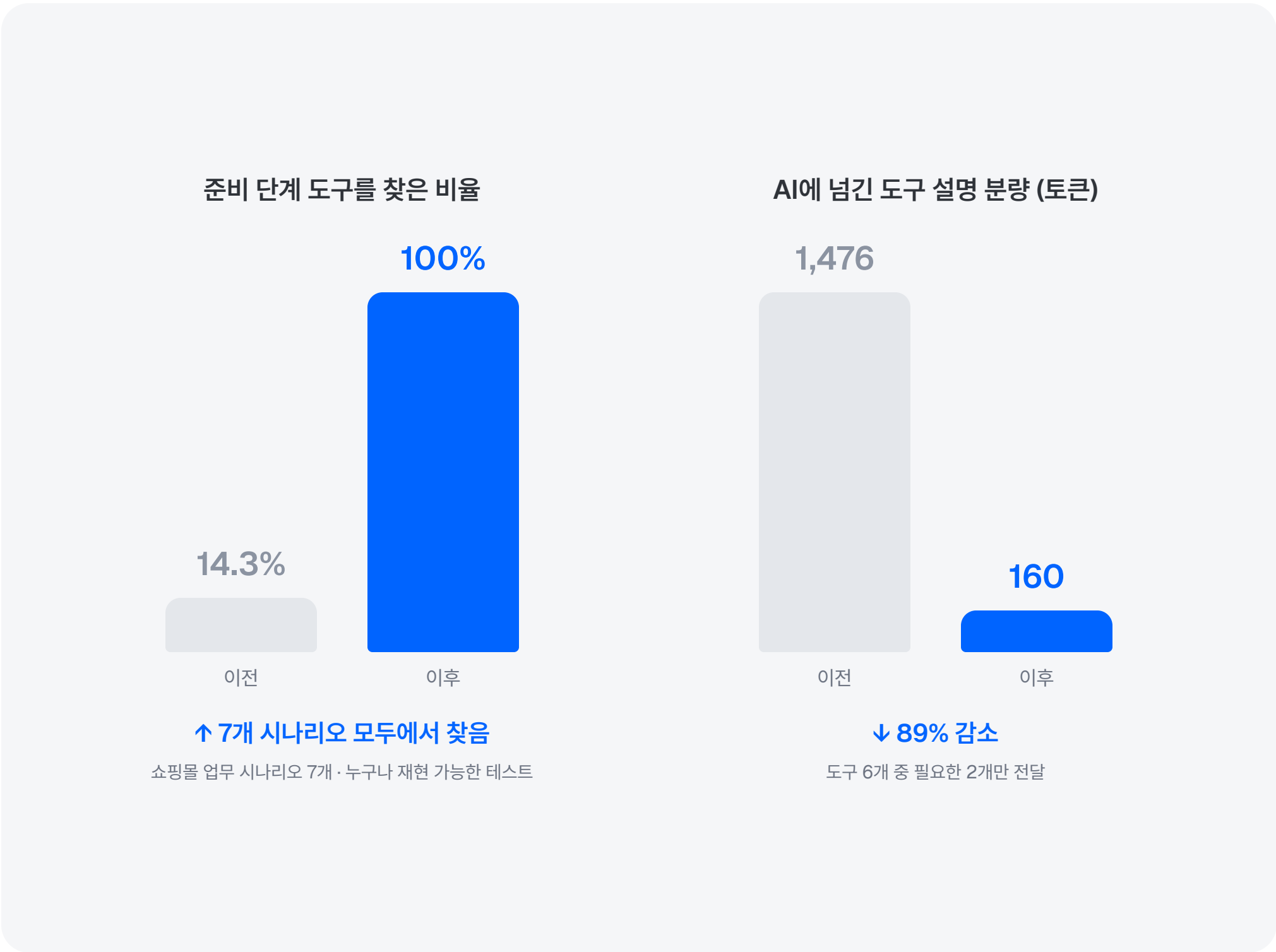
*v0.37.0 회귀 세트 · 커머스 체인 7개 · LLM 미사용 · 모집단 정확도 주장이 아님

해야 할 일의 도구만 찾던 방식이 놓치던 준비 단계 도구를 7개 시나리오 모두에서 찾아냈고, 실행
순서까지 맞춘 비율도 47.6%에서 100%가 됐습니다. 필요한 도구만 골라 넘기기 때문에 모델에
전달하는 스키마는 오히려 줄었습니다.

실서비스 적용 · XGEN 도구 결과 압축

도구 선택을 줄여도 사내 API 응답 JSON이 그대로 쌓이면 대화가 금방 한도를 넘었습니다.
HTTP 헤더를 버리고, JSON 목록은 첫 항목만 전체로 남기고 나머지는 식별 키만 남기도록
압축했습니다.

실제 XGEN API 응답 12건 기준 301K → 7.2K 문자로 줄었습니다 (-97.6%, v0.19.0).



결과 및 회고

재현 안 되는 주장은 스스로 내렸다



초기 벤치마크의 248개 도구 정확도 주장은 케이스별 결과가 남지 않아 다시 확인할 수 없었습니다.

그래서 릴리스 주장에서 빼고, 7개 케이스뿐이지만 누구나 같은 결과를 재현할 수 있는 회귀 세트로 바꿨습니다.

설계 변경은 모두 릴리스로 남겼다



코어를 표준 라이브러리만으로 유지하고, OpenAI·Anthropic·LangChain·MCP 어댑터는 선택 설치로 분리했습니다.

최근 180일 PyPI 다운로드 9,094회이고, GeekNews에 소개됐고 외부 개발자 4명이 fork했습니다.



8.21→4.54 6→3 0건 -37%

작업 하나에 AI를 부른 횟수 AI 호출 횟수 안 해 놓고 했다고 매번 붙는
평균 · 개발 환경 적용 중앙값 보고한 경우 · 60회 검사 기본 지시문 분량

XGEN 에이전트 하네스

에이전트가 일을 끝내게 만드는 실행 통제

하네스는 모델 바깥에서 도구 호출·권한·비용·완료 판정을 통제하는 계층입니다.
런타임의 21단계 파이프라인은 파트원이 설계했고, 저는 2026년 8월부터 실사용 기록을 분석해 하네스를 튜닝하고
외부 벤치마크로 평가 체계를 만들어 변경을 채택·기각했습니다. 파트는 LangChain·LangGraph로 만든 실행 경로를 자체 하네스로 옮기는 중이고, 저는
LangChain 노드와 LangGraph 챗봇을 만든 데 이어 튜닝·평가를 맡았습니다.

2026년 8월 ~ 현재

- 회사
- 튜닝 · 평가 설계
- 파이프라인 설계는 동료

비용 폭증, 거부 무시, 거짓 완료 보고

모델은 답을 했지만 일은 끝나지 않았습니다. 실사용 기록에서 서로 다른 세 가지 새는 곳을 찾았습니다.

한 턴이 끝나지 않고 입력 토큰이 250만 개를 넘었고, 사용자가 거부한 삭제를 다시 물었고, 샌드박스만 보고 삭제했다고 답했습니다.

프롬프트를 늘리는 대신, 이 세 가지를 모델 바깥의 하네스가 통제하도록 바꾸기로 했습니다.

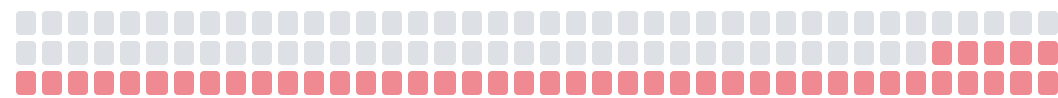
근거·실사용 기록 2026-09-16·09-23·실험실 재현 3/3·9월 3주 집계

실사용에서 찾은 세 가지 사건 (날짜가 서로 다른 별개 사건)

비용 · 09-16

2,524,884

요청 한 번에 AI가 읽은 분량 (토큰)



그중 같은 도구 45회 연속

권한 · 09-23

3번

거부한 삭제를 다시 물은 횟수

rm -rf 실행할까요?

거부

rm -rf 실행할까요?

거부

rm -rf 실행할까요?

거부

완료 판정

3회

실험실에서 거짓 “삭제했습니다” 재현 (3번 중 3번)

샌드박스
파일 삭제됨

사용자 PC
파일 그대로

개선 전 실제 요청 4,083건 중 **336건(약 8%)**이 도중에 멈춤 · 2026년 9월 3주간

하네스 AI 바깥에서 실행을 통제하는 장치

토큰 AI가 읽고 쓰는 글자 단위

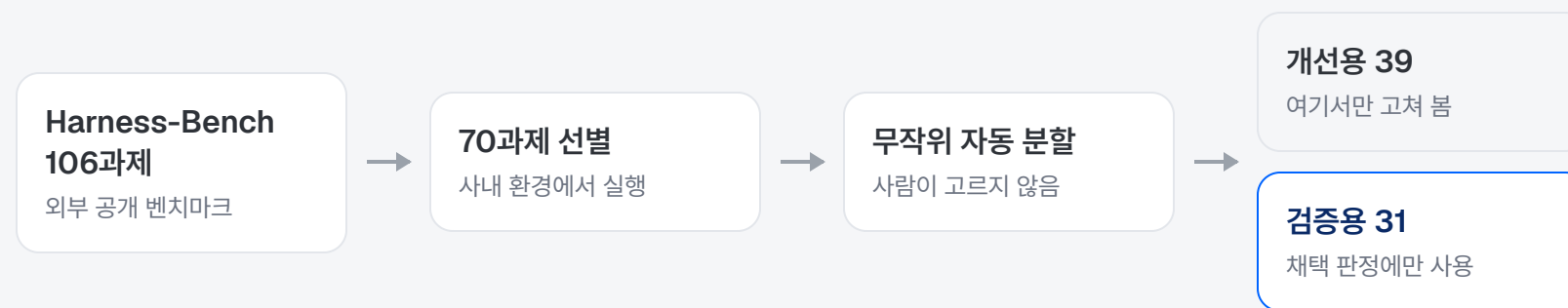
채택 기준을 먼저 정했다

직접 설계 · 벤치 연결 · 자동 분할 · 성적표

튜닝에 쓴 과제로 결과를 판정하면 그 과제에만 맞춘 변경이 채택됩니다.

외부 공개 벤치마크를 가져와 과제를 무작위로 자동 분할하고, 고쳐 보는 건 개선용에서만, 채택 판정은 검증용에서만 했습니다.

근거 · 평가 도구 커밋 기록 (2026-09-19 ~ 26)



채택 규칙

개선용에서 좋아지고, 처음 보는 검증용에서도 좋아질 때만 반영

점수 흔들림

같은 설정 4회 반복 범위 0.772 ~ 0.834

채점 오류

발견 후 7개 과제 다시 채점

검증용 과제

개선에 쓰지 않고 판정에만 쓰는 과제

벤치마크

같은 조건으로 성능을 비교하는 과제 모음

지시문 대신 구조로 바꿨다

“도구를 묶어 호출하라”는 지시는 38과제 중 한 번만 지켜졌습니다. 모델에게
부탁하는 방식은 재현되지 않았습니다.

그래서 파일 첨부, 빠른 경로, 반복 감지, 거부 차단처럼 하네스가 구조로 강제하는
변경만 남기고, 지시문과 파라미터 조정은 기각했습니다.

근거 · 실험별 비교 보고서와 커밋 기록 (2026-09)

채택 5 시스템이 강제하는 변경

01 첫 턴에 요청 파일 첨부
도구 호출 9.6 → 7.9회 · 점수는 유지

02 작은 작업은 짧은 경로 + 결과물 형식 검사
AI 호출 8.21 → 4.54회 · 초안 Codex, 적용·검증 직접

03 같은 호출 반복 감지
개선용 14건 중 13건, 검증용 3건 모두 잡음 · 오탐 0

04 거부한 동작 재시도 차단
확인 창 3번 → 1번

05 실행 기록 속 비밀번호·토큰 가림
2만 건 점검 · 노출 34 → 0

기각 4 지시문·설정값 조정

01 “도구를 묶어 호출하라” 지시
38과제 중 1번만 따름

02 요구사항 재확인 단계 추가
점수 하락 · AI 호출 39% 증가

03 멈춤 감지 장치
점수 하락 0.823 → 0.799

04 답변 무작위성 낮추기
늘 실패하던 37개 중 1개만 회복

AI 호출은 절반, 점수는 그대로

70개 과제에서 작업당 AI 호출 횟수가 평균 8.21회에서 4.54회와 4.67회로 줄었고, 중앙값은 6회에서 3회가 됐습니다.

첫 실행의 점수 상승(0.847)은 두 번째 실행(0.807)에서 재현되지 않았고, 두 번째 값은 같은 설정을 네 번 돌린 점수 범위 안이라 향상으로 주장하지 않습니다.

기준선 1회 실행 · 빠른 경로는 기본 꺼짐, develop 환경만 적용 · 21단계 파이프라인은 파트원 설계

작업당 AI 호출 횟수 · 70과제 평균



중앙값 6회 → 3회

평가 점수 · 70과제



같은 설정을 네 번 돌린 점수 범위 0.772 ~ 0.834

1차 0.847은 범위 위였지만 2차에서 재현 안 됨 → “나빠지지 않음”으로만 주장

0건

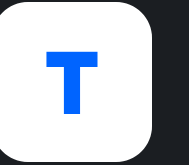
안 해 놓고 했다고 보고 · 60회 검사

3 → 1

거부한 뒤 다시 묻는 횟수

12/12

실제 업무형 과제 모두 성공



14B 11개 SFT 2024.05

AI Lab이 공개한
한국어 LLM POLAR

x2bee에 공개한
POLAR 모델 · 2024

제가 맡은 학습 단계
7B·14B 모델 공개

Open LLM Leaderboard
제출

LLM 파인튜닝

서빙만이 아니라 모델 학습도 한다

입사 직후 플래티어 AI Lab 4인 팀으로 SOLAR-10.7B 기반 14B 한국어 LLM POLAR를 만들어 공개했습니다.

베이스 모델과 DPO는 동료가, SFT 학습과 학습 스크립트는 제가 맡았습니다. 지금은 작은 결정 모델을 LoRA로 학습하고 있습니다.

2024년 4월 ~ 6월 · 2026년 9월 ~

회사 · 개인

SFT 학습 · 스크립트

AI Lab 4인 팀

팀 모델을 만들고 SFT를 맡았다

POLAR는 AI Lab 4명이 단계를 나눠 만들었습니다. 베이스 모델, SFT, DPO 모델을 차례로 공개하고 Open LLM Leaderboard에 제출했습니다.

저는 SFT를 맡아 POLAR-7B-14B SFT 모델을 학습해 공개했고, 그 전에 Mistral·Llama QLoRA로 학습 방식을 먼저 실험했습니다.

근거 · Hugging Face x2bee/POLAR-* 모델 카드(개발자 명단)와 커밋 기록 · 본인 계정 SuperPowerMz

2024.04 ~ 06

AI Lab 4인 · SFT 담당

POLAR

베이스
14B · SOLAR-
10.7B 기반

SFT
7B · 14B 공개

DPO
선호 데이터
1.23만 건

직접 TRL·DeepSpeed SFT 학습, POLAR-7B-SFT · 14B SFT 공개

직접 사전 실험 Mistral-7B · Llama-7B QLoRA

팀 x2bee에 POLAR 모델 11개 공개, Open LLM Leaderboard 제출

2026.09 ~

개인 · 단독

sev

데이터셋
결정 문제

LoRA 학습
Qwen3.5-4B

평가
라벨 과제 · 체스

Eikos 기반 결정 모델의 데이터셋, 학습, 서빙, 평가를 한 하네스로 관리

평가 절차를 결과보다 먼저 문서로 고정

학습에 쓰지 않은 과제(체스)로도 평가



323건

검색 엔진에 올린
내 코드 변경 · 14개월

68%

Rust 전환 초기 커밋 중
내 커밋 비율 (93/136)

3가지

한 번의 검색에 합친 기준
키워드 · 상품 의미 · 색상

10ms

검색 서버만의 응답 시간
검증 서버 20회 평균

커머스 상품 검색 엔진

키워드가 못 찾는 상품을 찾게 하기

커머스 고객사 상품 검색에서 “빨간 여성 롱코트” 같은 문장형·색상형 검색어는 키워드가 정확히 맞지 않으면 잘 찾지 못했습니다.
NestJS 검색 엔진에 벡터 검색을 합친 하이브리드 쿼리를 만들었고, 이후 Rust 검색 API에서 색인 파이프라인과 DB 계층을 맡았습니다.

2024년 7월 ~ 2026년 4월 · NestJS 검색 14개월 + Rust 전환

회사

하이브리드 쿼리 · 색인 직접

OpenSearch

X2BEE AI

전자신문 · 2024.09.24

플래티어, ‘엑스투비 AI’ 출시...“시맨틱 검색과 이미지 검색을 지원하는 AI 서치”

내 몫 · 같은 시기 시맨틱(벡터) 검색 쿼리 개발

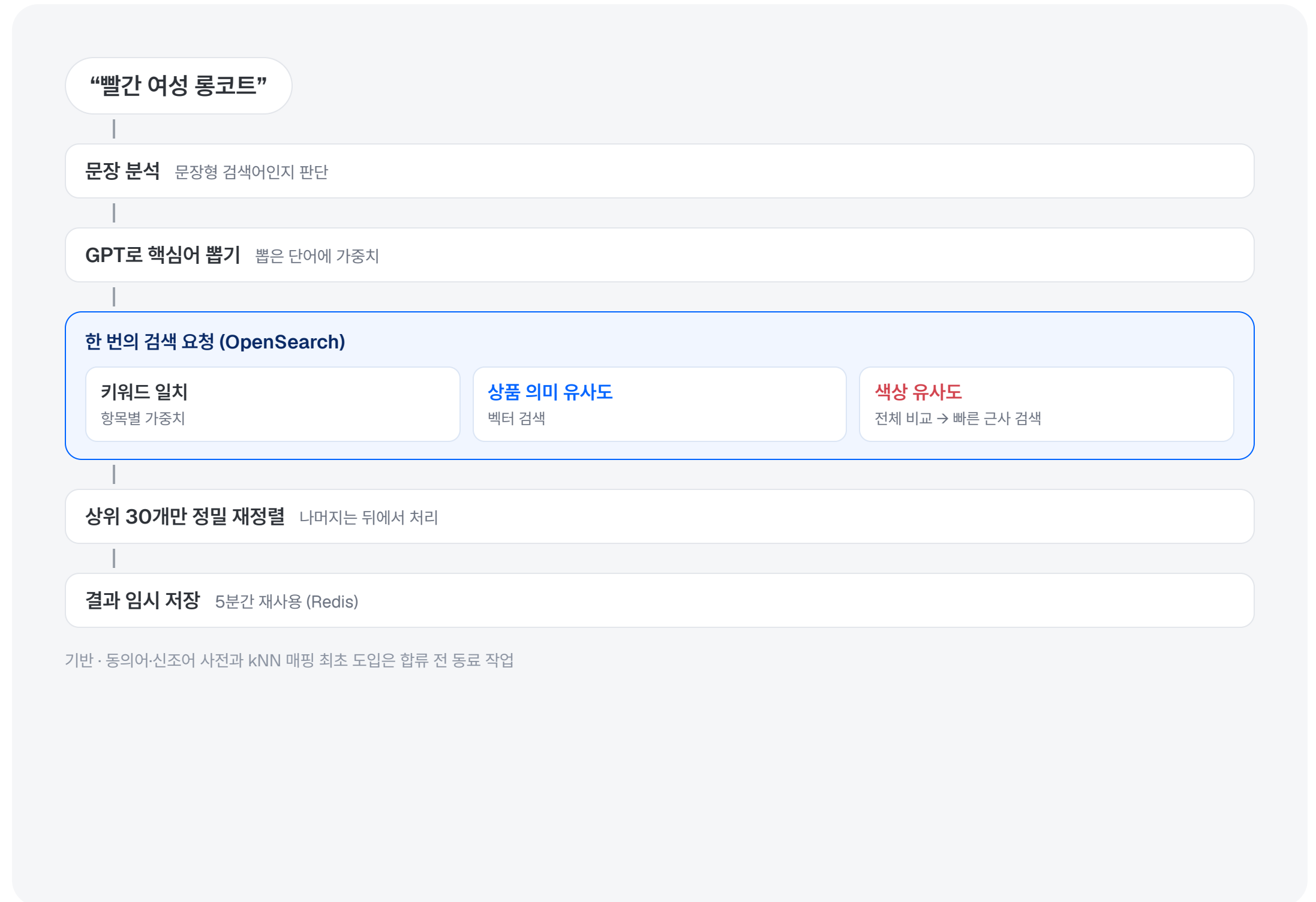
키워드·상품·색상을 한 쿼리로

직접 설계 · 커밋 323건 (2024.07 ~ 2025.09)

검색어를 먼저 분석해 자연어인지 판별하고, GPT가 뽑은 핵심어에 가중치를 준 뒤
키워드·상품 벡터·색상 벡터를 하나의 OpenSearch 쿼리로 합쳤습니다.

색상은 전체 상품과 일일이 비교하던 계산을 빠른 근사 검색(kNN)으로 바꿨고, 정밀
재정렬은 상위 30개만 바로 하고 나머지는 뒤에서 처리해 임시 저장했습니다.

근거 · 커밋 기록 2024-10 ~ 2025-01



색인은 다시 짜고, 느린 구간은 재서 찾았다

Rust 전환 초기 커밋의 68% · 색인·DB 부분 직접

Rust 검색 API로 옮기면서 색인 파이프라인을 스트리밍으로 다시 짰습니다.
Axum 뼈대와 이후 벡터 검색은 파트원 작업입니다.

외부 성능 시험에서 검색 응답이 느리다는 보고를 받고 구간별로 재 보니, 검색 API는 10ms였고 느린 구간은 그 앞단이었습니다.

색인 파이프라인 재설계

- 01

나눠서 색인하기

5만 건씩 흘려 넣고, 넣는 동안 갱신 멈춤
- 02

일괄 처리 구조는 7일 만에 견어냄

한 가지 방식으로 통일 · 코드 8,123줄 삭제
- 03

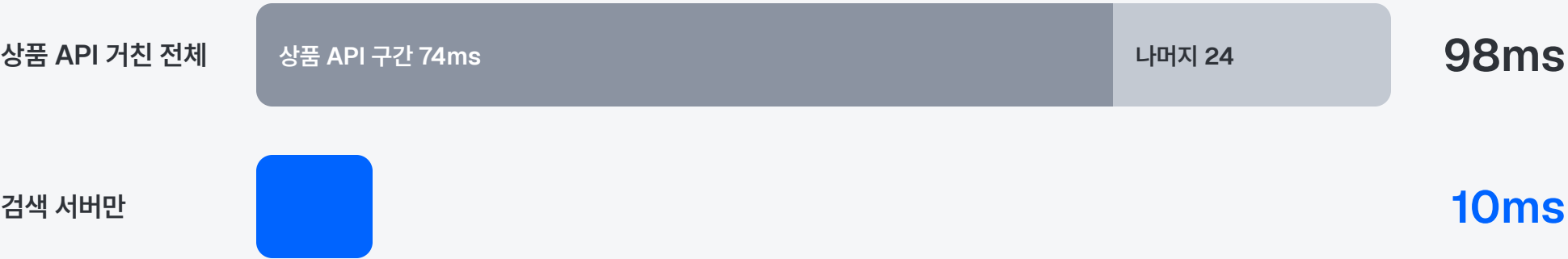
동시 색인 충돌 막기

전체 재색인과 수시 갱신이 겹치지 않게 잠금
- 04

검색 사전 반영 방식 교체

서버 파일 복사 → API 호출

응답 시간을 구간별로 재 보니 · 느린 곳은 검색 서버 밖



검증 서버 · 검색어 1개 · 20회 평균 (7~14ms) · 동시 500건에서도 중간값 13ms

제가 가져올 수 있는 세 가지

01

고객 현장과 폐쇄망에서도 LLM을 운영한다

현장 FDE · 서버 8곳을 Helm 차트 하나로 · 모델 실행 엔진 10종

FDE · XGEN · 하네스

02

수많은 사내 시스템에서 필요한 것만 찾는다

준비 단계까지 찾는 도구 검색 · 키워드와 의미를 합친 상품 검색

graph-tool-call · 커머스 검색

03

틀리면 안 되는 판단은 재현으로 확인한다

개선용·검증용 분리 · 누구나 다시 돌려 보는 명령 · 버린 실험도 기록

하네스 · graph-tool-call · 파인튜닝



GitHub
github.com/SonAlengine



기술 블로그 286편
infoedu.co.kr

손성준 · sonsj97@gmail.com